

智能错题本：基于大模型的错题管理与 AI 解答系统

SUM26001 编程课程期末项目报告

卓妍 (202320102053)

项目概述

问题陈述

学生在日常学习中会积累大量错题，传统的手动抄录方式存在以下痛点：

- 耗时费力**：每道错题需要手写抄录，效率极低，一道错题平均需要 5-10 分钟
- 整理困难**：抄在纸上的错题难以按学科、题型分类和检索，复习时翻找不便
- 缺乏深度理解**：抄完题目后往往不知道错在哪里、为什么错，缺少针对性的解析
- 复习不便**：纸质错题本容易丢失、难以携带，且无法快速搜索
- 公式难录入**：数学、物理等科目包含大量公式（分数、上下标、积分符号等），手抄极易出错且难以辨认

本项目旨在解决上述问题：**用手机拍一张错题照片，AI 大模型识别文字（含数学公式）、实时渲染 Markdown 和 LaTeX 公式、AI 给出详细解答、按学科和题型自动分类存储**——将错题管理的完整流程数字化、智能化，将单道错题的整理时间从 5-10 分钟压缩到 30 秒以内。

项目目标

- 拍照识别**：通过手机拍照上传错题图片，支持裁剪选取题目区域

2. **AI 大模型识别**：使用视觉大模型替代传统 OCR，准确识别文字和数学公式，输出 LaTeX 格式
3. **公式与 Markdown 渲染**：在识别结果、错题列表、AI 解答、打印输出四个场景中统一支持 LaTeX 公式和 Markdown 格式渲染
4. **AI 智能解答**：调用推理大模型，给出结构化解答（正确答案、解题思路、知识点、易错提醒）
5. **学科与题型管理**：支持自定义学科（名称 + 颜色）和题型分类，按学科整理错题
6. **搜索与打印**：按学科筛选、关键词搜索，一键生成 A4 打印稿
7. **移动端可用**：PWA 支持，可安装到手机桌面像原生 App 一样使用
8. **隐私保护**：所有数据存储在浏览器本地，无需服务器，保护用户隐私

已取得的进展

截至报告撰写时，项目已完成全部核心功能开发并部署上线：

- **在线体验地址**：<https://Zy1222dgbsw.github.io/error-notebook-app/>
- **GitHub 仓库**：<https://github.com/Zy1222dgbsw/error-notebook-app>
- **Git 提交次数**：30+ 次，完整记录了从初始框架到最终产品的迭代过程
- **代码规模**：约 2500 行（HTML + CSS + JavaScript）
- **功能模块**：拍照录入、AI 识别、公式渲染、AI 解答、学科管理、搜索打印、PWA 安装

经验教训

在开发过程中，我学到了以下重要经验：

1. **大模型优于传统 OCR**：Qwen3-VL 视觉大模型不仅能识别文字，还能理解公式结构并输出 LaTeX，彻底解决了传统 OCR 无法处理数学公式的问题
2. **多模型降级策略至关重要**：API 服务可能因模型下架、网络波动等原因不可用，设计 4 级降级链确保服务连续性
3. **渲染时序控制是前端难点**：KaTeX 公式渲染需要在 DOM 元素插入后执行，动态生成的内容需要统一的渲染管理
4. **CDN 选型要考虑地域**：jsdelivr 在国内被墙导致白屏，必须准备国内镜像降级方案

5. **JavaScript 字符串语法**：单引号字符串中不能包含实际换行符，必须使用 `\n` 转义——这个错误曾导致整个应用加载失败

技术方案

技术栈

层级	技术	选型理由
页面结构	HTML5	原生 Web 标准，支持摄像头调用（ <code><video></code> 、 <code><canvas></code> ）
样式	CSS3	响应式布局 + CSS 变量深色模式体系
逻辑	JavaScript (ES6+)	无框架依赖，代码清晰可读
AI 识别	硅基流动 API + Qwen3-VL-32B	视觉大模型，支持公式输出，识别准确率高
AI 解答	硅基流动 API + DeepSeek-V3	推理能力强，中文理解优秀
公式渲染	KaTeX 0.16.10	最快的 Web 数学公式渲染库
Markdown	marked.js 14.1.1	轻量级 Markdown 解析器
数据存储	localStorage	简单可靠，数据在用户设备，隐私安全
离线支持	Service Worker	PWA 离线缓存
版本控制	Git + GitHub	完整开发历史记录
部署	GitHub Pages	免费静态网站托管

AI 模型架构

识别模型（视觉大模型替代传统 OCR）

采用多模型降级策略，依次尝试直到成功：

优先级	模型	参数规模	说明
主模型	Qwen/Qwen3-VL-32B-Instruct	320 亿	识别精度最高，支持公式输出
备选 1	Qwen/Qwen3-VL-30B-A3B-Instruct	300 亿 (MoE)	平衡速度与精度
备选 2	Qwen/Qwen3-VL-8B-Instruct	80 亿	轻量级，响应速度快
备选 3	zai-org/GLM-4.5V	—	智谱视觉模型，多模态能力强

解答模型

模型	说明
deepseek-ai/DeepSeek-V3	DeepSeek 最新大模型，推理能力强，中文理解优秀

从 OCR 到大模型识别的技术演进

维度	旧方案	新方案
识别引擎	Tesseract.js (传统 OCR)	Qwen3-VL-32B (视觉大模型)
运行方式	浏览器端本地运行	云端 API 调用
公式支持	不支持	支持 LaTeX 格式输出
识别精度	一般 (复杂排版易出错)	高 (大模型理解上下文)
首次加载	需下载 30MB 语言包	无需下载, 直接 API 调用
AI 解答模型	Qwen2.5-7B-Instruct	DeepSeek-V3 (推理能力更强)
内容渲染	纯文本	Markdown + LaTeX 公式渲染

成果展示

功能模块

项目实现了从拍照到归档的完整错题管理流程：

- 拍照录入**：支持拍照、相册选择、拖拽上传、剪贴板粘贴四种方式；内置图片裁剪功能（四角拖拽调整）
- AI 大模型识别**：调用 Qwen3-VL 视觉大模型，识别文字并输出 LaTeX 公式格式；显示识别进度和当前调用的模型名称
- 编辑与预览**：识别结果可编辑，支持「编辑/预览」模式切换；预览模式实时渲染 Markdown 和 LaTeX 公式
- AI 智能解答**：调用 DeepSeek-V3 模型，输出结构化解答（正确答案、解题思路、知识点、易错提醒），全部支持公式渲染
- 学科与题型管理**：自定义学科（名称 + 颜色 + 题型），按学科分类整理错题
- 搜索与筛选**：按学科筛选 + 关键词搜索

- 7. **打印输出**：勾选错题 → 一键生成 A4 打印稿，含 AI 解答和答题空白区，支持公式渲染
- 8. **PWA 安装**：支持添加到手机桌面，全屏无浏览器地址栏，离线可用

代码统计

文件	功能	行数
index.html	页面结构 + CDN 引入	~250 行
style.css	响应式样式 + 深色模式 + 渲染样式	~500 行
app.js	全部业务逻辑 (AI 识别 + 渲染 + AI 解答)	~1500 行
manifest.json	PWA 配置	~30 行
sw.js	Service Worker 离线缓存	~50 行
README.md	项目文档	—
.gitignore	版本控制配置	—
report.qmd	本报告源文件	—
合计		~2500 行

瓶颈记录

在开发过程中，我遇到了多个技术瓶颈。以下按照「核心挑战 → 卡住的地方 → 做出的调整 → 解决方案」的格式逐一记录。

瓶颈 1：传统 OCR 无法识别数学公式

我们面临的核心挑战：使用 Tesseract.js 浏览器端 OCR 引擎识别错题图片中的文字，包括数学公式。

我们在哪里卡住了：Tesseract.js 是基于传统 OCR 技术的引擎，它只能识别纯文本字符，无法理解数学公式的结构。对于上下标（如 x^2 ）、分数（如 $\frac{a}{b}$ ）、积分符号（如 $\int_0^1 f(x)dx$ ）等数学符号，Tesseract.js 要么识别为乱码，要么完全忽略。这导致数学、物理等科目的错题识别结果完全不可用——而这些恰恰是最需要数字化的科目。

为应对瓶颈所做的调整：放弃 Tesseract.js，转而使用视觉大模型替代传统 OCR。接入硅基流动 API，调用 Qwen3-VL-32B 视觉大模型进行文字识别。视觉大模型不仅能识别文字，还能理解图片中的公式结构并输出 LaTeX 格式。

我们找到的解决方案：

- 设计多模型降级策略：Qwen3-VL-32B $\rightarrow$ Qwen3-VL-30B-A3B $\rightarrow$ Qwen3-VL-8B $\rightarrow$ GLM-4.5V，确保单个模型不可用时仍能提供服务
- 优化识别提示词，明确要求模型输出 LaTeX 格式公式（行内用 `$...$`，独立用 `$$...$$`）和 Markdown 结构
- 实现图片压缩（Canvas $\rightarrow$ JPEG），将大图压缩到 1MB 以下再发送 API 请求
- 添加 API 超时处理和错误重试机制

瓶颈 2：LaTeX 公式在动态生成的内容中不渲染

我们面临的核心挑战：在 OCR 识别结果、错题列表、AI 解答、打印输出四个场景中统一支持 LaTeX 公式和 Markdown 渲染。

我们在哪里卡住了：KaTeX 的 `renderMathInElement` 函数需要在 DOM 元素插入到文档后才能执行。但在实际应用中，错题列表是通过 `innerHTML` 动态生成的，AI 解答是异步加载的，打印页面是在新窗口中生成的——这些场景的渲染时机各不相同，导致公式经常无法正确渲染。最初只有 AI 解答弹窗的公式能正常显示，其他场景都是纯文本。

为应对瓶颈所做的调整：

- 封装统一的渲染函数 `renderContentToElement(element, text)`：先使用 marked.js 解析 Markdown，再将 HTML 插入 DOM 元素，最后调用 `renderMathInElement` 渲染公式
- 封装 `renderTextToHTML(text)` 函数：返回渲染后的 HTML 字符串，用于 `innerHTML` 拼接场景

3. 新增 `renderErrorListContent()` 函数：在错题列表渲染完成后，使用 `querySelectorAll` 遍历所有需要渲染的容器，统一调用 `KaTeX`

我们找到的解决方案：使用 `setTimeout` 延迟渲染 (50ms)，确保 DOM 元素已插入文档后再调用 `renderMathInElement`。在四个场景中统一使用相同的渲染逻辑和公式定界符配置（`$$...$$`、`$...$`、`\(...\)`、`\[...\]`），确保任何场景下公式都能正确显示。

瓶颈 3：CDN 被墙导致页面白屏

我们面临的核心挑战：通过 CDN 加载 `KaTeX`、`marked.js` 等第三方库，确保页面正常加载。

我们在哪里卡住了：最初使用 `jsdelivr.net` 作为 CDN 源，但在国内访问时该域名被墙，导致 `KaTeX` 和 `marked.js` 无法加载，页面直接白屏——所有依赖这两个库的功能（公式渲染、Markdown 解析）完全失效。

为应对瓶颈所做的调整：将 CDN 源从 `jsdelivr` 切换为 `npmmirror`（国内镜像），并设计多 CDN 自动降级机制：优先加载 `npmmirror`，失败后尝试 `cdnjs`，最后尝试 `unpkg`。

我们找到的解决方案：实现 CDN 加载失败检测——在 `<script>` 标签上添加 `onerror` 回调，当主 CDN 加载失败时自动插入备用 CDN 的 `<script>` 标签。这样即使用户网络环境复杂，也能保证关键库的加载。

瓶颈 4：旧版视觉模型被硅基流动下架

我们面临的核心挑战：维持 AI 识别功能的稳定运行，确保用户随时可以使用。

我们在哪里卡住了：项目最初使用的视觉模型（如 `Qwen/Qwen2.5-VL-7B-Instruct` 等）被硅基流动平台下架，API 调用返回 404 错误，导致 OCR 识别功能完全失效。

为应对瓶颈所做的调整：将所有视觉模型替换为硅基流动当前可用的 `Qwen3-VL` 系列模型。同时设计了 4 级降级策略，当一个模型不可用时自动切换到下一个，大幅提高了服务的可用性。

我们找到的解决方案：定期检查硅基流动的模型列表，使用最新的 `Qwen3-VL-32B-Instruct` 作为主模型。同时在代码中实现 `tryWithModel` 递归降级机制，当某个模型返

回错误时自动尝试下一个模型。

瓶颈 5：JavaScript 语法错误导致应用完全无法加载

我们面临的核心挑战：在升级 OCR 提示词后，确保应用正常运行。

我们在哪里卡住了：在修改 OCR 提示词时，将多行文本直接写入单引号字符串中，导致 JavaScript 语法错误。单引号字符串中不能包含实际换行符，浏览器解析 `app.js` 时抛出 `SyntaxError: Invalid or unexpected token`，整个应用加载失败，用户看到的是空白页面或错误提示。

为应对瓶颈所做的调整：使用 `\n` 转义序列替代实际换行符，确保字符串在单引号内正确表示多行文本。同时使用 `node --check` 命令验证 JavaScript 语法正确性。

我们找到的解决方案：

1. 将 OCR 提示词中的实际换行符替换为 `\n` 转义序列
2. 删除重复的函数定义（`renderErrorListContent` 被意外定义了两次）
3. 移除不必要的 `stream: false` 参数
4. 将 `temperature` 从 0.1 调整为 0.7（更标准的值）
5. 增强 API 错误处理，显示具体的错误信息方便调试

瓶颈 6：手机拍照图片过大导致 API 超时

我们面临的核心挑战：支持手机拍照上传错题图片，确保在各种网络条件下都能顺利完成 AI 识别。

我们在哪里卡住了：现代手机摄像头拍摄的图片通常超过 5MB（base64 编码后更大），直接发送给硅基流动 API 会导致请求超时或被拒绝。部分用户的网络上传速度有限，大图片传输耗时过长，用户体验很差。

为应对瓶颈所做的调整：在发送 API 请求前，使用 Canvas API 对图片进行压缩。将图片最大宽度限制为 1280 像素，质量因子设为 0.75，输出 JPEG 格式。压缩后的图片通常在 200–500KB 之间，大幅减少了传输时间和 API 处理时间。

我们找到的解决方案：实现 `compressImage(dataUrl, maxWidth, quality, callback)` 函数，在图片大小超过 1MB 时自动触发压缩。压缩过程在浏览器端完成，不消耗服务器资源。

瓶颈 7：Service Worker 缓存旧页面导致功能不更新

我们面临的核心挑战：使用 PWA 技术实现离线缓存，同时确保用户始终能获取最新版本。

我们在哪里卡住了：Service Worker 会缓存所有静态资源（HTML、CSS、JS），当开发者更新代码后，用户浏览器中仍然加载旧的缓存版本，导致新功能不生效、旧 Bug 依然存在。用户需要手动清除缓存或强制刷新才能看到更新。

为应对瓶颈所做的调整：在 Service Worker 中实现缓存版本管理——每次更新代码时递增缓存版本号，在 `activate` 事件中清理旧版本缓存。

我们找到的解决方案：

1. 定义全局缓存版本变量 `CACHE_VERSION`
2. 在 `install` 事件中预缓存核心资源
3. 在 `activate` 事件中遍历所有缓存，删除不属于当前版本的旧缓存
4. 使用 `clients.claim()` 确保新的 Service Worker 立即生效

瓶颈 8：测试 API Key 按钮点击无视觉反馈

我们面临的核心挑战：在设置弹窗中点击「测试 API Key」按钮后，期望看到模型可用性测试结果，但实际没有任何视觉反馈。

我们在哪里卡住了：HTML 元素 `<div id="apiTestResult" style="display:none">` 使用了内联样式 `display:none`，其 CSS 优先级高于 JS 设置的 `element.hidden = false`。即使 JS 把 `hidden` 属性设为 `false`，内联 `display:none` 仍然生效，结果区始终隐藏，导致用户点击按钮后看不到任何反馈。

为应对瓶颈所做的调整：放弃通过 `hidden` 属性控制显隐，改用 `element.style.display = 'block'` 显式操作内联样式。同时为 `showToast` 函数增加 `info` 类型（蓝色提示），延长显示时间到 4 秒。

我们找到的解决方案：

1. 使用 `result.style.display = 'block'` 显式设置内联样式，绕过优先级问题
2. 为 `showToast` 增加 `info` 类型支持，CSS 添加 `.toast.info` 蓝色样式
3. `info` 类型 toast 显示 4 秒（普通 2.5 秒），给用户足够时间查看测试进度

4. 自动滚动到结果区，让测试进度清晰可见

瓶颈 9：打印功能被浏览器拦截

我们面临的核心挑战：点击「打印选中」按钮后没有反应，无法生成打印预览。

我们在哪里卡住了：原实现使用 `window.open()` 打开新窗口打印，但现代浏览器对 `window.open` 的拦截策略非常严格——即使在用户点击事件中调用，也可能被浏览器的弹窗拦截器拦截。此外，`formatAIAnswer` 函数内部有 `setTimeout` 调用，尝试在新窗口查找 `$('.answerContent')` 元素，但该元素在新窗口中根本不存在，可能抛出错误。

为应对瓶颈所做的调整：放弃 `window.open` 方案，改用隐藏 `iframe` + `iframe.contentWindow.print()` 打印。这种方式完全在当前页面内完成，不触发任何弹窗拦截。

我们找到的解决方案：

1. 创建零尺寸 (0×0) 的 `iframe` 元素并附加到 `body`
2. 将打印 HTML 写入 `iframe` 的文档
3. 调用 `iframe.contentWindow.print()` 触发打印
4. 打印完成后延迟 1 秒移除 `iframe`，清理 DOM
5. 打印页面的 AI 解答使用内联的 `marked.parse` 直接转换，不调用可能出错的 `formatAIAnswer`
6. 添加加载状态 `toast` (`info` 类型) 和错误处理

瓶颈 10：移动端打印失效 + 浏览器缓存导致旧代码运行

我们面临的核心挑战：在手机上点击「打印选中」按钮仍无反应；同时 OCR 识别错误提示的模型名 `Pro/Qwen/Qwen2-VL-7B-Instruct` 是旧模型，但实际 `main` 分支已经更新为新模型 `Qwen/Qwen3-VL-32B-Instruct`。

我们在哪里卡住了：经过排查发现两个独立的、互相叠加的问题：

1. **iOS Safari 等移动浏览器不支持 0×0 隐藏 `iframe` 的 `contentWindow.print()`。**
iOS 要求 `iframe` 至少有一定的可见区域，且对程序调用的 `print()` 有额外限制

2. **GitHub Pages + 浏览器 HTTP 缓存**使得用户在第一次访问后，即使服务端代码已经更新，浏览器仍会优先使用本地缓存的 `app.js`，导致「代码已更新但用户看不到」的情况

为应对瓶颈所做的调整：

1. **打印方案改为三段式降级**：首选 Blob URL + iframe，失败时降级到「在当前页面显示打印内容」方案——保存当前 body，替换为打印 HTML，配「 点击打印 / 保存为 PDF」按钮和「← 返回错题本」按钮
2. **引入版本检测机制**：在 `app.js` 顶部定义 `const APP_VERSION = '15'`，每次启动检查 localStorage 中的版本号 `errorNotebook_appVersion`，不一致时自动 toast 提示「检测到新版本」
3. **升级资源 URL 版本参数**：`style.css?v=14` → `?v=15`，强制浏览器绕过 HTTP 缓存重新下载

我们找到的解决方案：

1. **打印功能**（移动端 + 桌面端通用）：
2. 优先使用 Blob URL (`new Blob([html])`) + `URL.createObjectURL`) + 隐藏 iframe 触发打印
3. iframe 加载完成后调用 `openPrintInCurrentPage()`，在当前页面显示打印内容
4. 打印页面顶部添加「← 返回错题本」浮动按钮，点击后恢复原页面并重新初始化
5. **缓存破除双重保险**：
6. URL 参数 `?v=15` 让浏览器认为是新资源
7. localStorage 版本号检测让用户主动确认已更新
8. **错误处理**：捕获所有可能的异常并显示友好的 toast 提示

关键教训：

- **PWA / Web 应用的版本管理至关重要**——必须设计版本检测机制，否则用户永远在用旧代码
- **移动端与桌面端的行为差异**——iOS Safari 对 iframe、window.open、print() 都有更严格的限制
- **HTTP 缓存是无声的杀手**——`v=N` 参数是简单但有效的缓存破坏手段

瓶颈 11：多部署环境 + 打印 iframe onload + 主题切换两击

我们面临的核心挑战：三个互相叠加的问题：(1) Codebuddy 部署环境与 GitHub 独立，用户使用的是旧代码；(2) 打印按钮在 PC 和手机端都没有反应；(3) 初次切换深色/白天模式需要点两次。

我们在哪里卡住了：

1. **Codebuddy 部署是独立的早期环境**——早期创建的 Codebuddy 部署 (5715cd46...app.codebuddy.work) 与 GitHub 仓库完全分离，git push 不会影响其上的代码。用户通过该链接重新安装后，看到的仍是旧代码，导致 OCR 错误、测试按钮无反应等问题持续存在
2. **iframe 的 onload 在 opacity:0 状态下不触发**——iOS Safari 对不可见的 iframe 有严格限制，如果 iframe 被设置为 opacity:0，其 onload 事件永远不会触发，导致后续 openPrintInCurrentPage 永远不被调用，用户看到「打印按钮无反应」
3. **主题切换逻辑缺陷**—— dataset.theme 初始为 'auto'，第一次点击时 cur='auto'，next = cur === 'dark' ? 'light' : 'dark' 总是得到 'light'。如果用户系统是浅色模式，视觉上无变化，必须再点一次才能切换到深色

为应对瓶颈所做的调整：

1. 引导用户使用 GitHub Pages 版本 (<https://Zy1222dgbsw.github.io/error-notebook-app/>)，并同步更新安装指南中的链接
2. 彻底移除 iframe 步骤，printSelectedErrors 直接调用 openPrintInCurrentPage，避免 onload 不触发的陷阱
3. 主题切换时使用 window.matchMedia('(prefers-color-scheme: dark)').matches 检测系统真实模式

我们找到的解决方案：

1. **链接统一：**所有 btnCopyLink 复制链接和 <code class="install-link"> 显示链接改为 GitHub Pages URL
2. **打印极简化的：**直接替换 document.body.innerHTML 为打印内容，配固定定位的打印/返回按钮

3. **主题智能检测**： `cur='auto'` 时读取系统偏好作为 `actualMode`，确保第一次点击就是真正的切换
4. **添加切换反馈**：切换后显示 toast「已切换到深色/浅色模式」

Git 版本控制

本项目通过 Git 进行版本控制，提交历史完整记录了开发过程。以下是关键提交节点：

提交	内容	阶段
初始提交	搭建 HTML/CSS/JS 项目框架	v0.1
feat: 裁剪 UI	实现图片裁剪功能	v0.2
feat: OCR 识别	集成 Tesseract.js OCR	v0.3
feat: AI 解答	接入 AI 解答功能	v0.4
feat: 学科管理	学科管理与 localStorage	v0.5
feat: 深色模式 + PWA	深色模式与 PWA 支持	v0.6
feat: 题型分类	自定义题型系统	v0.7
feat: 大模型替换 OCR	Qwen3-VL 系列视觉大模型	v0.8
feat: 公式 Markdown 渲染	KaTeX + marked.js 全场景渲染	v0.9
feat: DeepSeek-V3 升级	AI 解答模型升级	v1.0
fix: OCR 语法错误修复	修复提示词语法错误	v1.0.1
docs: 添加期末报告	report.qmd + report.pdf	v1.0.2
fix: 修复测试 API Key	style.display 修复 + info toast	v1.0.3
fix: 修复打印按钮	iframe 打印方案	v1.0.3
fix: 打印+版本检测	移动端打印 + 缓存破除	v15
fix: 打印+主题+链接	极简打印 + 主题智能检测 + 链接统一	v16
feat: 大模型替换 OCR	Qwen3-VL 系列视觉大模型	v0.8
feat: 公式 Markdown 渲染	KaTeX + marked.js 全场景渲染	v0.9
feat: DeepSeek-V3 升级	AI 解答模型升级	v1.0
fix: OCR 语法错误修复	修复提示词语法错误	v1.0.1

完整提交历史可在 GitHub 仓库查看：<https://github.com/Zy1222dgbsw/error-notebook-app/commits>

总结与反思

项目收获

- 技术层面**：学会了大模型 API 调用、视觉模型集成、LaTeX 公式渲染、Markdown 解析、PWA 离线缓存等前端技术
- 工程层面**：理解了版本控制（Git）的重要性，体验了迭代式开发流程，积累了 Bug 定位和修复经验
- 产品层面**：从用户需求出发设计功能，关注移动端体验，考虑隐私和可用性
- 调试能力**：通过 7 个主要瓶颈的排查与解决，积累了丰富的前端调试经验

解决的问题

本项目解决了学生手动抄录错题的低效问题。通过"拍照 → AI 大模型识别 → Markdown/公式渲染 → AI 解答 → 学科+题型分类"的自动化流程，将原本需要 5-10 分钟的错题整理压缩到 30 秒以内。更重要的是，AI 不仅给出答案，还解释解题思路，帮助学生真正理解错题。公式的自动识别和渲染解决了数学、物理等科目错题数字化的最大障碍。

未来改进方向

- 添加错题导出功能（PDF/图片格式）
- 支持手写体 OCR 识别（需要更专业的模型）
- 添加错题复习模式（随机出题、错题统计）
- 实现多设备同步（引入后端数据库）
- 添加错题标签和难度评级
- 支持更多学科专属功能（如英语听力、化学方程式）

结语

这个项目完整地展示了"发现问题 → 分析需求 → 选择技术 → 逐步实现 → 迭代优化"的软件工程流程。从最初的 Tesseract.js 传统 OCR，到最终接入 Qwen3-VL 视觉大模型 + DeepSeek-V3 推理模型 + KaTeX 公式渲染，每一次升级都解决了真实的用户痛点。7 个技术瓶颈的排查与解决过程，不仅锻炼了我的调试能力，也让我深刻理解了前端工程中渲染时序、CDN 选型、API 降级策略等关键问题。项目在满足课程评估标准的同时，也产出了一个真正可用的工具。